

Spring Security — Interview Notes

1. Authentication vs Authorization

Authentication → verifies who the user is.

Authorization → verifies what the authenticated user is allowed to access.

Authentication = Identity

Authorization = Permission

2. JWT Authentication

JWT (JSON Web Token) is a signed token commonly used for **stateless authentication**.

JWT structure:

```
Header.Payload.Signature
```

Request:

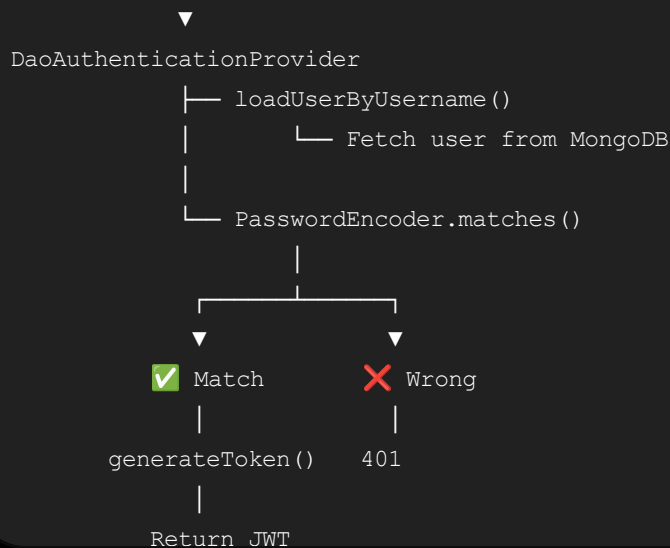
```
http
```

```
Authorization: Bearer eyJhbGci...
```

Important: JWT is signed by default, not encrypted.

3. Login Flow

```
POST /api/auth/login
{ username, password }
|
▼
JwtAuthFilter
|
└─ No JWT → skip
|
▼
permitAll()
|
▼
AuthenticationManager.authenticate()
|
```



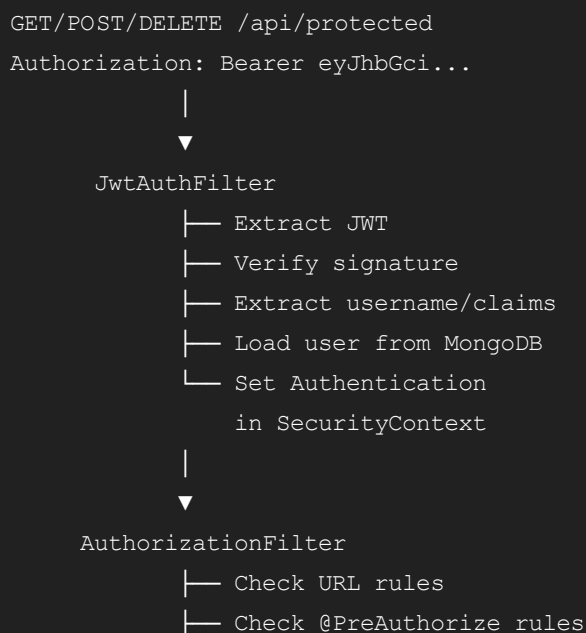
Key components

- `AuthenticationManager` → coordinates authentication.
- `DaoAuthenticationProvider` → performs username/password authentication.
- `UserDetailsService` → loads user details.
- `PasswordEncoder` → verifies the password hash.
- `generateToken()` → creates JWT after successful authentication.

Interview statement

"`AuthenticationManager` delegates authentication to `DaoAuthenticationProvider`, which loads the user through `UserDetailsService` and verifies the password using `PasswordEncoder`."

4. Protected Request Flow



- |
- | — ☒ Authorized → Controller
- | — ☐ Unauthenticated → 401
- | — ☐ Insufficient permission → 403

5. SecurityContext

`SecurityContext` stores the current user's `Authentication`.

```
SecurityContext
  ↓
Authentication
  |— Principal/User
  |— Authorities/Roles
  |— Authentication status
```

After successful JWT validation:

Java

```
SecurityContextHolder
    .getContext()
    .setAuthentication(authentication);
```

This allows the rest of the application to know **who is making the request**.

6. Authorization

Security rules can be configured using `SecurityFilterChain`.

Example:

Java

```
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.anyRequest().authenticated()
```

Method-level authorization:

Java

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) {
```

```
}
```

```
@PreAuthorize
```

Checks authorization **before the method executes**.

7. 401 vs 403

Status	Meaning
401 Unauthorized	User is not authenticated / authentication failed
403 Forbidden	User is authenticated but lacks required permission

Easy memory

401 → Who are you?

403 → I know who you are, but you can't do this.

8. Important Spring Security Components

Component	Responsibility
<code>SecurityFilterChain</code>	Defines security configuration
<code>JwtAuthFilter</code>	Extracts and validates JWT
<code>AuthenticationManager</code>	Coordinates authentication
<code>DaoAuthenticationProvider</code>	Username/password authentication
<code>UserDetailsService</code>	Loads user
<code>PasswordEncoder</code>	Password hashing/verification
<code>SecurityContext</code>	Stores current authentication
<code>Authentication</code>	Represents authenticated identity
<code>GrantedAuthority</code>	Roles/permissions
<code>@PreAuthorize</code>	Method-level authorization

9. OAuth 2.0

OAuth 2.0 is an authorization framework, not a token format.

Typical flow:

```
User
↓
Client Application
↓
Authorization Server
↓
Access Token
↓
Resource Server/API
```

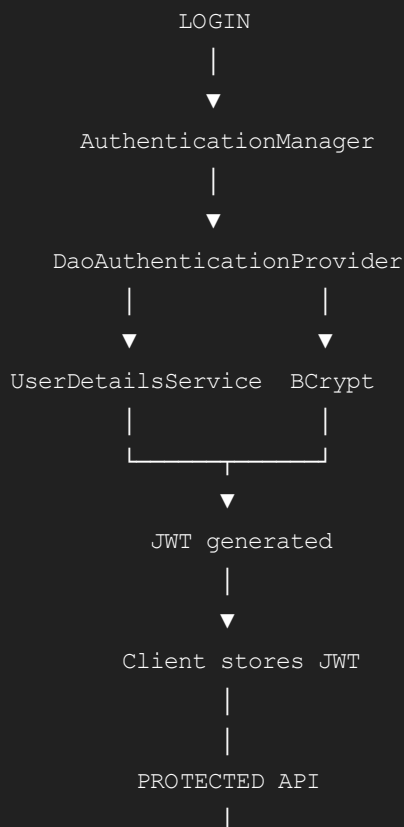
Important distinction

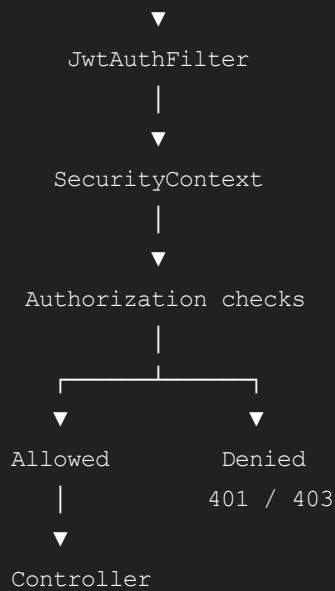
OAuth2 = Authorization framework

JWT = Token format

OAuth2 can use JWT as an access token, but **OAuth2 does not require JWT**.

10. Complete Mental Model





Interview Must-Remember

1. **Authentication** → identity
2. **Authorization** → permissions
3. **JWT** → stateless authentication token
4. `AuthenticationManager` → delegates authentication
5. `DaoAuthenticationProvider` → username/password authentication
6. `UserDetailsService` → loads user
7. `PasswordEncoder` → verifies password
8. `JwtAuthFilter` → validates JWT on protected requests
9. `SecurityContext` → holds authenticated user
10. **401** → authentication problem
11. **403** → authorization problem
12. **OAuth2** → authorization framework
13. **JWT ≠ OAuth2**
14. `@PreAuthorize` → method-level authorization